HFDN Enterprise LIS + RIS Platform

Professional software architecture and implementation roadmap

Your specification describes more than a Laboratory Information System (LIS/LIMS). It is a multi-centre healthcare operations platform combining laboratory management, radiology workflows, patient services, financial management, referral management, equipment integration and enterprise reporting.

My recommendation is to build it as a **modular, API-first Django platform**, with a premium Bootstrap 5 web interface, a Flutter mobile application, PostgreSQL as the primary database, and a dedicated local integration service for laboratory analysers and imaging systems.

The most important architectural decision is to separate the central healthcare application from the devices that operate inside your laboratory and radiology network. This lets you deploy the main platform to Railway while keeping analyser communication, local device protocols and other hardware-dependent services on computers inside HFDN's centres.

1. LIS / LIMS

Patient registration, test ordering, sample collection, barcodes, batching, analyser results, review, authentication, controlled result release, quality control and laboratory reporting.

2. RIS + PACS integration

Radiology worklists, modality assignments, examination start/end, structured reporting, radiologist authentication and integration with imaging archives such as Orthanc.

3. Enterprise operations

Billing, discounts, tariffs, HMO and credit accounts, referral commissions, inventory, notifications, audit trails, multi-centre administration and analytics.

1. Recommended technology stack

Layer	Recommended technology	Purpose
Backend	Django + Django REST Framework	Business logic, APIs, permissions and workflows
Database	PostgreSQL	Reliable relational data and transaction processing
Web interface	Bootstrap 5, premium admin template, JavaScript	Responsive staff, management and portal dashboards
Mobile	Flutter	Patient, referral physician and authorised staff applications
Authentication	Django authentication + secure session/token strategy	Role-based and portal-specific access
Background jobs	Celery + Redis	Notifications, exports and asynchronous processing
File storage	S3-compatible object storage	Registration forms, referral documents and reports
Deployment	GitHub + Railway	Version-controlled builds and hosted application services
Imaging	DICOM-compatible PACS, such as Orthanc	Imaging storage and viewing integration

Layer	Recommended technology	Purpose
Device gateway	Local Windows service with a local configuration UI	Analyser communication, buffering and forwarding

A critical deployment distinction: Railway should host the central application, but it should not be expected to communicate directly with USB-connected devices or private-network analysers at HFDN. Those devices require a secure on-site gateway.

2. System architecture

Presentation layer

Bootstrap 5 web application · Flutter mobile app

Front Desk · Laboratory · Radiology · Accounts · Management · Patient/Doctor Portals

Django REST API

Authentication · RBAC · Centre access · Workflow engine · Audit logging

PostgreSQL

Patients, visits, orders, results, invoices, payments and audit records.

Background services

Celery, Redis, notifications, exports and scheduled jobs.

Secure centre integration layer

Local device gateway · LIS analyser interfaces · RIS/PACS · Printers and scanners

Encrypted outbound communication, local queues, acknowledgements and retry handling.

The central application remains the source of truth for patient identity, visits, orders, billing and authorised results. The local gateway handles device-specific communications and must not bypass central permissions or silently overwrite validated clinical data.

3. Design the application as separate Django modules

Do not build the entire application in one Django app or one giant set of views. Use separate modules with defined responsibilities, shared security rules and controlled interactions between them.

Django app	Main responsibilities
`accounts`	User accounts, authentication, password reset and MFA
`organization`	Centres, departments, centre assignments and operating settings
`rbac`	Roles, permissions, access policies and permission auditing
`patients`	Patient profiles, UHID, demographics, duplicate detection and documents
`visits`	Registration, centre-specific Visit IDs, referrals and visit history
`catalog`	Departments, service groups, tests, modalities, tariffs and test parameters
`orders`	Test orders, amendments, cancellations, add-ons and order history
`billing`	Invoices, payments, receipts, discounts, refunds and credit accounts
`samples`	Sample IDs, barcode printing, recollection, batching and sample tracking
`workflows`	Configurable workflows, step transitions, queues and timelines

Django app	Main responsibilities
`laboratory`	Department receiving, processing, result entry, review and authentication
`radiology`	RIS worklists, modality scheduling, examination lifecycle and reports
`pacs`	DICOM metadata, imaging references and PACS connectivity
`referrals`	Referral physicians, onboarding, referral links, earnings and settlements
`portals`	Patient, physician and corporate client portals
`notifications`	Email, SMS, WhatsApp and message templates
`integrations`	Analyser connections, gateway registration, inbound messages and mappings
`inventory`	Reagents, consumables, stock movements and equipment records
`reporting`	Financial, clinical, operational and management reports
`audit`	Security events, clinical history and immutable business audit records

Start with a **modular monolith**, rather than dozens of independent microservices. It is easier to build, test, deploy and maintain while the platform is being developed. Extract services later only when a genuine scaling or isolation requirement emerges.

4. Multi-centre access and role-based security

The system must distinguish between what a user can do and where that user can do it.

For example, a laboratory scientist may have `enter_lab_results` permission at Enugu HQ and NOWAS, while a cashier may have `process_payments` only at Independence.

Permission model

Use four dimensions:

- **Role:** Laboratory Scientist, Lab Manager, Front Desk, Cashier, Accountant, Radiographer, Radiologist, Administrator, Referral Doctor or Patient.
- **Permission:** `register_patient`, `edit_demographics`, `approve_discount`, `authenticate_result`, `release_result`, `edit_bill`, `settle_referral` and similar granular actions.
- **Centre assignment:** Which centres a user can access.
- **Scope:** Whether access applies to their own records, assigned department, assigned centre, multiple centres or the entire organisation.

A user interface checkbox should never be the only permission control. The Django API must independently enforce every permission.

Suggested permissions for sensitive actions

Permission	Example restriction
`can_modify_patient_data`	Demographic changes require permission and an audit entry
`can_edit_bill`	Add or remove tests through a recorded adjustment process
`can_approve_discount`	Approval restricted to authorised users and limits
`can_review_results`	Review results within assigned laboratory scope
`can_authenticate_results`	Authentication restricted to designated qualified personnel
`can_release_results`	Release only when every applicable release condition is satisfied

Permission	Example restriction
`can_change_referral_doctor`	Mandatory reason, note and supporting document
`can_settle_referral`	Financial settlement requires proof of payment
`can_view_financial_reports`	Restricted financial information and export permissions

Audit metadata

For significant events, record:

- User and action
- Timestamp in UTC, displayed in the Africa/Lagos timezone
- Centre and department
- Relevant patient, visit, order or result identifier
- Previous and new values where applicable
- Source IP address and browser/device information
- Reason, approval and supporting evidence where required

IP addresses and browser strings are useful audit metadata, but neither proves the user's identity or physical location. Avoid collecting precise geolocation unless it is necessary, disclosed and appropriately authorised.

5. Patient identity, Visit IDs and Sample IDs

Your specification establishes three different identifiers. They must remain separate in the database.

Patient identifier · persistent

UHID

One patient identity across all centres and visits.

Phone number assists duplicate matching; the UHID itself should remain stable if the patient changes their number.

Visit identifier · one per registration/visit

NWS0000001

NOWAS centre prefix plus a seven-digit, zero-padded sequence.

Example: Independence uses IND0000001. Maintain a separate sequence for each centre.

Sample identifier · one per collected sample

21300001

Centre digit + department digit + service-group digit + five-digit sequence.

Illustrative example: centre 2, department 1, service group 3, sequence 00001.

Database and generation rules

Create separate records for `Patient`, `Visit`, `Sample` and `TestOrder`.

- A patient can have many visits.
- A visit can have many test orders.
- A visit can have many samples.

- A sample can be associated with the appropriate order or orders according to the configured collection requirements.
- A recollection creates a new sample record linked to the original sample and order; it must not erase the original collection history.

Generate Visit IDs and Sample IDs using database-backed counters and atomic transactions. Do not rely on counting existing records or using an ordinary Python increment, because simultaneous registrations can generate duplicates.

Add database uniqueness constraints and retry-safe generation logic.

Important design point: your sample-ID structure has only five sequence digits. That permits 99,999 nonzero sequence values per centre/department/service-group combination if numbering starts at `00001`. Define what happens when a sequence reaches its limit before production deployment.

6. Patient registration, billing and result-release controls

The front desk should use a single registration workspace with clear tabs or sections.

Patient registration screen

Patient: UHID / demographics

Visit: Centre / Visit ID

Orders: Tests / tariff / prices

Documents: Referral / registration form

Release eligibility: Required documents present, applicable workflow complete, results authenticated, outstanding financial restrictions resolved and release permission granted.

Mandatory registration documents

Provide a “Scan Document” or “Upload Document” feature for the referral document or registration form.

The system should:

1. Upload the file to protected object storage.
2. Associate it with the patient and the correct visit.
3. Record document type, uploader, timestamp and file integrity metadata.
4. Verify that at least one acceptable required document exists for that visit.
5. Prevent result dispatch if the document requirement is not satisfied.

The release button should be disabled visually and the backend must also reject the release request. A disabled button alone is not a security control.

Keep the document's status independent from the sample and result statuses so that staff can see exactly what is blocking release.

Tariffs, billing and payment

Implement a tariff engine supporting Standard, HMO-specific, corporate and other approved price schedules.

Each tariff entry should be associated with a test, centre or applicable pricing scope, currency, effective dates and optional outsourcing cost.

For example:

Test	Standard tariff	HMO 1 tariff	XYZ tariff
CBC	■5,500	■4,900	■7,200

Prices are illustrative from your specification's pricing concept, not verified current HFDN prices.

When a test has no configured price, the system should show an explicit warning and prevent accidental completion of billing under an undefined price. Any permitted override should be controlled and audited.

For bill edits, never delete or silently overwrite a completed payment. Record the amendment, price difference, refund or additional payment and the identity of the authorising user.

For HMO or credit clients, distinguish an invoice or credit receipt from evidence that money has actually been received. The payer should be the organisation, with the patient and service details linked to the transaction.

Discounts should follow a request → approval or rejection → payment completion workflow. The approver should receive an in-app notification, with email/SMS as appropriate.

7. Laboratory workflow engine — the core of the LIS

Your requirement for configurable workflows is one of the most important features in the entire system. A fixed sequence of Django views will become difficult to maintain when microbiology, cytology, histology and routine laboratory tests require different processes.

Recommended workflow design

Create a workflow builder with configurable steps, allowed transitions, responsible roles, required fields, document requirements and conditions.

Normal laboratory workflow

1. Patient registration
2. Payment confirmation or approved credit arrangement
3. Sample collection and barcode generation
4. Batch creation and dispatch
5. Department receiving
6. Sample processing and activity logging
7. Result entry
8. First review / result validation
9. Result authentication by authorised personnel
10. Controlled result release

Alternative workflows

Cytology workflow

Registration → Payment → Collection → Batching → Cytology pending dashboard → Cytology processing → Result entry → Review → Authentication → Release.

Department receiving is skipped only when the configured workflow explicitly permits it.

Microbiology workflow

Registration → Payment → Collection → Batching → Department receiving → Culture processing → Incubation and observations → Result entry → Review → Authentication → Release.

Incubation durations, repeat observations and reporting steps should be configurable per test and laboratory policy.

Histology workflow

Registration → Payment → Specimen collection → Accessioning → Grossing and tissue processing → Embedding/sectioning → Microscopy → Report entry → Review → Authentication → Release.

These are proposed starting templates. HFDN's laboratory leadership should approve the actual workflow, including any skipped steps, before the templates are activated.

Workflow builder interface

The administrator should be able to:

- Create a workflow and assign it to a test or service group.
- Add, remove, reorder and configure steps using drag-and-drop.
- Assign roles and departments to each step.
- Define required fields, document requirements and conditional transitions.
- Configure escalation rules and expected turnaround times.
- Preview the workflow before activation.
- Version and publish workflow changes without corrupting tests already in progress.

Store workflows as versioned definitions in the database. When an order is created, bind it to the active workflow version. A later workflow change must not silently rewrite the history of existing orders.

The workflow engine should enforce state transitions server-side. For example, a result cannot be authenticated before review if review is a mandatory preceding step.

Sample tracking and recollection

Each sample should have a lifecycle and complete activity history.

Activity	Information to record
Collection	Collector, collection time, centre and sample type
Barcode	Sample ID, barcode value and print history
Batching	Batch ID, included samples and dispatch time
Receiving	Receiving staff, department, time and acceptance/rejection
Processing	Processing steps, observations and timestamps
Recollection	Original sample, reason, new sample ID and collector
Rejection	Reason, authorising staff and corrective action

A recollection should create a new sample linked to the original order. Mark the earlier sample as rejected, unsuitable or superseded according to the reason. Keep its audit trail intact.

Turnaround time and timeline

Build a unified event timeline from registration through release. Record both event timestamps and the users or devices responsible.

Provide:

- Total turnaround time (TAT).
- Time spent in each workflow stage.
- Pending work by centre, department and responsible role.
- Overdue orders and configurable escalation alerts.
- Recollection rates and sample rejection rates.
- Separate timestamps for result entry, review, authentication and release.

Define TAT rules per test or service, including whether the clock pauses for documented reasons.

8. Laboratory result templates and test configuration

The system should support both structured result entry and formatted clinical reports.

Test catalogue configuration

Each test should support:

- Test code, name, department and service group.
- Specimen type and collection requirements.
- Required sample volume, container and handling instructions.
- Applicable workflow and workflow version.
- Turnaround target.
- Tariff eligibility and outsourcing settings.
- Result template and reference range configuration.
- Units, decimal precision, qualitative or quantitative result types.
- Critical-result rules and any required comments.
- Instrument mappings and external analyser codes.

Template builder

For a CBC, for example, the result template might contain haemoglobin, white blood cell count, platelet count and other configured parameters, each with its own units and reference ranges.

Provide field types such as:

- Numeric result with unit and reference interval.
- Text or free-text interpretation.
- Single-select or multi-select result.
- Positive/negative or other controlled qualitative values.
- Tables, differential counts and calculated parameters.
- Comments, specimen notes and approved report sections.

Support age- and sex-dependent reference intervals where appropriate, and allow laboratory-approved ranges to be configured by test and applicable population. Do not hardcode clinical reference ranges into the frontend.

Templates must be versioned. Every released report should retain the template and reference-range versions used when the result was authenticated.

9. RIS and PACS: radiology workflow

Radiology should be a dedicated module sharing patients, visits, billing and access control with the LIS.

Radiology examination lifecycle

1. Patient registration and radiology order
2. Successful payment or authorised credit
3. RIS worklist creation
4. Radiographer opens examination with `Start Exam`
5. Modality performs study; images are sent to PACS where configured

6. Radiographer records `End Exam`
7. Study becomes available for reporting by an appropriately assigned radiologist
8. Radiologist prepares and signs the report
9. Report is released through the controlled result-release process

The `Start Exam` and `End Exam` actions should be recorded in the examination history. The reporting workflow must check the actual server-side examination state rather than merely hiding the result-entry button.

Modality-specific templates

Create configurable report templates for CT, X-ray, mammography, ultrasound and MRI. Each template can contain indication, technique, findings, impression and other approved fields specific to the examination.

Enforce your requirement that only the assigned reporting radiologist, or a properly authorised substitute under a controlled policy, can edit an unauthenticated report. Once signed or released, changes should require a formal amendment or addendum with an audit trail.

DICOM and PACS integration

For imaging, plan to support:

- DICOM study and series metadata.
- Modality worklist integration where the equipment supports it.
- DICOM image storage and retrieval.
- Study status reconciliation between the modality, RIS and PACS.
- Secure image-viewer access through an approved viewer.
- Report-to-study association and study identifiers.
- Failure queues and reconciliation for studies that do not arrive.

Orthanc is a practical PACS integration option for evaluation. The RIS should reference imaging studies instead of storing large image files as ordinary Django uploads. PACS availability, backups, access controls and storage capacity require their own operational plan.

10. Referral physician and corporate client portals

Your referral management requirement combines patient referrals, clinical visibility and financial settlement. Treat these as separate but linked capabilities.

Physician onboarding

1. An authorised administrator registers the physician and records the required professional and contact details.
2. The system creates an invitation with a short-lived, single-use activation token.
3. The physician receives an activation link by email.
4. The physician verifies the account, sets a password and completes any required identity checks.
5. The system activates the account and grants the appropriate portal permissions.

Do not email a permanent password in plain text. Use secure activation links and password-reset flows.

Direct referrals and front desk validation

Provide a referral-registration form in the physician portal. Each referral should generate a unique, difficult-to-guess token or reference that the patient can present at the front desk.

The front desk should validate the referral, confirm the patient's identity and finalise registration without creating a duplicate patient unnecessarily.

The referring physician should be selectable during ordinary registration. Include:

- The actual referring physician.
- `SELF` for a walk-in patient.
- `DEFAULT REF DOC` when the referrer is unknown or not yet onboarded.

Changing a referral doctor must require a reason, note and mandatory supporting attachment, with the previous attribution retained in the audit history.

Referral commissions and settlements

Create a ledger-based referral commission engine linked to the original registration, test order, invoice and financial transactions.

For each eligible order:

- Calculate commission using the configured percentage and eligible revenue basis.
- Record the calculation rule and amount at the time it is earned.
- Adjust or reverse the commission when a test is cancelled, refunded or replaced.
- Display the original and revised amounts with their transaction history.
- Prevent the same payment or order event from creating duplicate commissions.

For example, if a ■5,000 CBC generates a 10% commission, the initial commission is ■500. If the CBC is cancelled and replaced with an eligible ■3,000 LFT, the system must reverse the original commission and record the new ■300 commission. The physician's ledger should show both events.

Commission eligibility, the revenue basis, deductions, tax treatment and payout rules should be approved by HFDN management and configured explicitly.

Withdrawal workflow

Implement the following statuses:

`Requested` → `Processing` → `Paid/Settled`

with `Declined` as an alternative outcome.

Enforce the seven-day withdrawal rule on the server. Notify the Accounts Head and Manager when a request is submitted. An authorised finance user verifies the request, records the actual payout and uploads mandatory proof of payment with a note before marking it settled.

The physician should see the request status, settlement history, commission ledger and eligible referral history. Access to detailed clinical results should be separately authorised and governed by patient consent, applicable law and HFDN policy.

Corporate clients should have their own portal for authorised patient listings, credit balances, invoices, statements, payments and results where disclosure is permitted.

11. Notifications and patient portal

Create a notification service that can deliver messages through email, SMS and WhatsApp according to the patient's selected channels.

The front desk should be able to select one or more preferred communication channels during registration. Store communication preferences and consent where required.

Recommended notification events

Event	Intended recipient
Patient registration	Patient
Referral received	Referral physician, where appropriate
Payment receipt	Patient or authorised payer
Discount request/decision	Requesting staff and approver
Sample recollection required	Relevant staff; patient if needed
Result authenticated	Relevant authorised staff
Result ready	Patient, if authorised and consented
Referral withdrawal request	Accounts Head and Manager
Referral payout settled	Referral physician
Integration failure	IT or designated integration administrator

Message templates should support variables such as `{{ patient_name }}`, `{{ centre_name }}`, `{{ visit_id }}` and `{{ result_link }}`. Validate template variables before publishing a template.

Security recommendation: do not send permanent passwords or sensitive clinical results in ordinary messages. Send a secure portal link, use short-lived access tokens, and require authentication where appropriate. Notifications should contain the minimum information needed.

The patient portal can provide:

- Profile and visit history.
- Payment receipts and invoices.
- Result availability and secure result viewing.
- Download of authorised reports.
- Notification preferences.
- Support and correction requests.

A portal login should not automatically grant access to every result associated with a phone number. Establish and verify the patient's identity, and implement appropriate controls for minors, guardians and shared phone numbers.

12. Enterprise reporting and financial dashboards

Build reporting on the same audited transactional records used by the operational modules. Do not maintain separate editable totals that can drift away from the underlying payments and orders.

Recommended management dashboard

Financial — Revenue and collections: Gross charges, discounts, refunds, net revenue and outstanding credit.

Clinical operations — Workload and turnaround: Pending tests, overdue work, released reports and TAT.

Centres — Centre performance: Revenue, volume, service mix and operational comparisons.

Referral network — Physicians and partners: Referral volume, eligible commissions and payout liabilities.

Report catalogue

The reporting module should support:

- Revenue by patient, referral physician, corporate partner, centre, date, department, service group and test.
- Daily, weekly, monthly and custom-period analysis.
- Gross charges, discounts, collections, refunds, net receipts and outstanding receivables.

- HMO and credit-client statements.
- Referral commission accruals, reversals and payouts.
- Patient demographics, gender-based statistics and service utilisation.
- Department productivity and test turnaround.
- Sample rejection, recollection and workflow bottlenecks.
- Radiology study volumes by modality and reporting status.
- Inventory usage, wastage and expiry where inventory tracking is enabled.

Define each financial metric precisely. For example, revenue earned, cash collected and invoice value are different measures and should not be combined under a single ambiguous label.

Allow authorised users to export Excel and PDF reports. Apply the same centre-level and role-level access restrictions to exported reports as to on-screen dashboards. Log sensitive exports and avoid placing unnecessary patient identifiers in management reports.

For larger data volumes, use background jobs for report generation and add database indexes and pre-aggregated reporting tables only after measuring query performance.

13. Laboratory analyser integration — recommended approach

This part deserves a separate integration architecture. Your requirement is to configure analysers from a user interface without changing application source code for every new machine.

The recommended solution is a **local Windows integration gateway** installed at each laboratory centre.

13.1 Which technology should run on the local PC?

I recommend Python for the integration engine, with a service manager appropriate for Windows and a local web interface.

Component	Recommendation
Protocol engine	Python with a modular adapter interface
Device configuration UI	Localhost web application using Bootstrap 5
Service execution	Windows Service, managed through a supported service wrapper
Local queue	SQLite-backed durable queue or equivalent reliable local storage
Central connection	HTTPS REST API with gateway-specific credentials
Configuration	Validated, versioned device profiles
Logging	Rotating local logs, message IDs and central monitoring
Packaging	Versioned Windows installer and signed releases where feasible

The local UI can run on `127.0.0.1` for configuration from the gateway PC itself. If multiple authorised workstations need access, provide a separately configured LAN-accessible interface with authentication and network restrictions. Do not expose an unauthenticated configuration page to the laboratory network.

13.2 The integration workflow

1. **Register the gateway.** An administrator creates a gateway identity and assigns it to a centre and department.
2. **Add a device.** Select the manufacturer, model, interface type, connection parameters and supported protocol profile.
3. **Test connectivity.** Check the serial port or TCP endpoint, communication parameters and basic message exchange.

4. **Map codes.** Link the analyser's test codes and result codes to the central catalogue. Configure units, flags and parameter mappings.
5. **Validate sample identity.** Match the analyser's barcode or sample identifier to an authorised central order and the correct patient/visit.
6. **Process incoming messages.** Parse, validate and store the original message; identify duplicates, errors and unsupported codes.
7. **Submit results.** Send results through the secure API, preserving the source device, message ID, timestamps and raw-message reference.
8. **Reconcile.** Receive a central acknowledgement and retry failed messages without creating duplicate results.

13.3 Support multiple analyser protocols

Do not write one giant parser with manufacturer-specific conditions scattered throughout the code.

Define a common adapter contract such as:

- `connect()`
- `disconnect()`
- `health_check()`
- `receive_message()`
- `parse_message()`
- `map_test_codes()`
- `submit_result()`
- `acknowledge_message()`

Each device adapter implements the capabilities actually supported by its analyser and protocol.

Potential interface families include ASTM-style laboratory messaging, HL7 v2, serial communication, vendor-specific TCP protocols and proprietary middleware. DICOM belongs to the imaging side rather than being a universal laboratory analyser protocol.

The actual protocol, order-download capability, bidirectional communication, licensing requirements and supported message formats must be verified for each specific analyser model and software configuration. Do not assume that all Sysmex, Mindray or Abbott devices support the same interface.

13.4 Result safety and exception handling

Automated results must not bypass the laboratory's validation and authentication rules.

The gateway should:

- Preserve the original message for traceability.
- Validate the sample ID, test mapping, units and message structure.
- Detect duplicate transmissions.
- Quarantine unmapped or suspicious results.
- Flag results that do not match an active order.
- Prevent silent overwrites of existing results.
- Record device timestamps and gateway receipt timestamps separately.
- Support retry, reconciliation and administrator review.
- Avoid treating a communication acknowledgement as clinical result authentication.

A robust design uses an idempotency key based on the gateway, device/message identity and relevant order/sample context. If the same message is transmitted twice, the central system should not create two separate clinical results.

13.5 Offline operation

If the internet connection fails, the gateway should retain eligible messages in an encrypted or appropriately protected local queue and transmit them when the connection returns.

The gateway must not assume that an offline result has been accepted centrally until the central API confirms acceptance. Define queue limits, disk monitoring, retry backoff, clock synchronisation and an operational alert for prolonged disconnection.

Before live use, test duplicate delivery, malformed messages, lost acknowledgements, service restarts, expired credentials and interrupted connectivity.

14. Database model and key relationships

The database should represent healthcare operations explicitly rather than storing an entire workflow inside a single patient record.

Logical relationships

- Organization operates Centres.
- Centre receives Visits.
- Patient attends many Visits.
- Patient has Patient Documents.
- Visit contains Test Orders.
- Test defines Test Orders.
- Visit generates Samples.
- Test Order is collected for by Samples.
- Sample has Sample Events and can be included in Batches.
- Test Order produces Results.
- Result records Result Events.
- Visit is billed by Invoices.
- Invoice receives Payments.
- Referral Doctor makes Referrals.
- Referral generates Commission Ledger entries.
- Centre hosts Gateways.
- Gateway manages Devices.
- Device receives Integration Messages.
- Test Order follows Workflow Instances.

These are logical relationships. The implementation will need additional entities for users, centre assignments, permissions, tariffs, invoice lines, refunds, result versions, workflow steps, review decisions, radiology studies, document access, notification deliveries and integration mappings.

Database design rules

- Use UUID primary keys for internal entities where appropriate, while retaining the required human-readable UHID, Visit ID and Sample ID.

- Enforce business identifier uniqueness at the database level.
- Use foreign keys and transactions for related financial and clinical operations.
- Use decimal fields for monetary values; never floating-point arithmetic for financial calculations.
- Store all important timestamps in UTC.
- Use explicit status fields and validated transitions instead of inferring status from whether a related record exists.
- Retain historical tariff, template and workflow versions.
- Use soft cancellation or reversal for clinical and financial records where appropriate, with tightly controlled deletion of data that can legitimately be removed.

15. Security, privacy and clinical governance

An enterprise healthcare application needs controls beyond ordinary login and role-based menus.

At minimum, include:

- MFA for administrators and privileged users.
- Secure password hashing, session management, password resets and account lockout/rate limiting.
- HTTPS for all external communications.
- Secrets managed through deployment environment variables or a proper secrets manager.
- Centre-level and object-level access checks in every relevant API.
- Encryption and access restrictions for documents, backups and sensitive stored data.
- Audit trails for clinical results, financial changes, user administration and document access.
- Tested backup restoration and disaster recovery procedures.
- Dependency scanning, vulnerability remediation and controlled software releases.
- Monitoring for suspicious logins, unusual data access and repeated failed integration attempts.

For Nigeria, have qualified legal and compliance advisers assess the Nigeria Data Protection Act 2023 and other applicable healthcare, professional and data-retention requirements. Establish policies for consent, patient access, correction requests, retention, breach response and permitted disclosures before production launch.

Clinical governance must also define who is qualified to enter, review, authenticate and release each category of result. The software should enforce those approved policies rather than deciding clinical authority solely from job titles.

16. Deployment architecture for GitHub and Railway

Your intended development workflow is appropriate: develop locally, push tested changes to GitHub, and deploy from GitHub to Railway.

Local development

Django · Bootstrap · Flutter · local PostgreSQL · automated tests

↓

GitHub repository

Feature branches · pull requests · tests · reviewed changes

↓

Railway staging

Build · migrate · smoke test · validate configuration

↓

Railway production

Approved release · PostgreSQL · HTTPS · monitoring · backups

Recommended repository structure

```
hfdn-platform/
■■■ backend/
■   ■■■ config/
■   ■■■ apps/
■   ■   ■■■ accounts/
■   ■   ■■■ organization/
■   ■   ■■■ rbac/
■   ■   ■■■ patients/
■   ■   ■■■ visits/
■   ■   ■■■ catalog/
■   ■   ■■■ orders/
■   ■   ■■■ billing/
■   ■   ■■■ samples/
■   ■   ■■■ workflows/
■   ■   ■■■ laboratory/
■   ■   ■■■ radiology/
■   ■   ■■■ referrals/
■   ■   ■■■ integrations/
■   ■   ■■■ notifications/
■   ■   ■■■ reporting/
■   ■■■ manage.py
■   ■■■ requirements.txt
■■■ web/
■   ■■■ templates/
■   ■■■ static/
■■■ mobile/
■   ■■■ Flutter application
■■■ gateway/
■   ■■■ adapters/
■   ■■■ local_ui/
■   ■■■ service/
■■■ tests/
■■■ docs/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

This is a suggested structure; the exact placement of Django apps and the web interface can be adjusted as the codebase evolves.

Production configuration checklist

1. **Separate environments:** local, staging and production should have separate credentials and databases.
2. **Environment variables:** configure `SECRET_KEY`, database URL, allowed hosts, trusted origins, email credentials, object-storage credentials and integration settings outside source control.
3. **Static assets:** configure Django static-file collection and an appropriate production serving strategy.
4. **Database migrations:** commit migrations to GitHub and apply them in a controlled deployment step. Never casually reset a production database to resolve a migration error.
5. **Background jobs:** deploy separate worker and scheduled-job processes when needed; do not run long jobs inside ordinary web requests.
6. **Health checks:** implement application health and readiness checks, including database connectivity.
7. **Files:** use persistent object storage for patient documents and reports rather than an ephemeral application filesystem.
8. **Backups:** enable database backups and regularly test restoration.
9. **Release controls:** use a staging deployment and smoke tests before promoting changes to production.
10. **Rollback:** document code rollback and database recovery procedures, recognising that a database schema change may not be safely reversible.

Railway is a reasonable starting point for hosting the central Django application. However, confirm the applicable service limits, storage requirements, regional availability, contractual terms and data-protection requirements before storing production patient data there. A production clinical platform also needs a tested disaster recovery plan beyond relying on a successful deployment.

17. Implementation phases and acceptance criteria

I would implement the system in the following order. Each phase should deliver a testable, deployable increment rather than waiting until every feature is complete.

Phase 01 — Foundation and infrastructure

Establish the repository, Django project, PostgreSQL, deployment pipeline, authentication, RBAC, centre assignments, audit logging and basic administration.

Acceptance: secure login, centre switching, enforced API permissions, automated tests and successful staging deployment.

Phase 02 — Patient registration and billing

Build UHID and Visit ID generation, duplicate checks, document uploads, test catalogue, tariffs, invoices, payments, receipts, discount approvals and bill amendments.

Acceptance: concurrent registrations cannot create duplicate IDs; payments reconcile; missing prices and documents are handled correctly.

Phase 03 — Core LIS workflow

Implement collection, sample barcodes, batches, inter-centre transfer, department receiving, recollection, processing, result templates, review, authentication and controlled release.

Acceptance: complete sample-to-result traceability and server-enforced workflow transitions.

Phase 04 — RIS and PACS integration

Build modality worklists, examination lifecycle, radiology reporting, study association and the initial PACS connection.

Acceptance: the radiology workflow prevents unauthorised reporting and reconciles studies, reports and status changes.

Phase 05 — Referral and patient portals

Add physician onboarding, referral validation, commission ledger, settlement requests, patient results access and corporate-client access.

Acceptance: commission adjustments are traceable; portal users cannot access another patient's data without authorisation.

Phase 06 — Local analyser gateway

Develop the local service, device configuration UI, protocol adapters, code mappings, durable queues and central API integration.

Acceptance: verified device communication, duplicate protection, offline recovery and reconciliation with real instrument messages.

Phase 07 — Enterprise reporting and hardening

Complete management dashboards, exports, notifications, performance tuning, backup recovery drills, security testing, user training and pilot deployment.

Acceptance: signed-off user acceptance testing, operational monitoring, tested recovery and documented release procedures.

These phases can overlap where interfaces and dependencies are clear, but clinical workflow, billing integrity, identity management and audit logging should be foundational rather than postponed.

18. Testing strategy

Before production use, implement automated tests for the highest-risk business rules.

Test category	Examples
Identity	Duplicate patients, changed phone numbers, concurrent Visit ID generation
Permissions	Cross-centre access, unauthorised result access, finance permissions
Billing	Duplicate payments, refunds, discount approvals, bill amendments
Samples	Barcode uniqueness, recollection links, batch reconciliation
Clinical workflow	Missing documents, skipped steps, review and authentication restrictions
Radiology	Start/end exam enforcement, radiologist assignment and signed-report amendments
Referrals	Duplicate commissions, cancellation reversals and settlement evidence
Integrations	Repeated messages, lost acknowledgements, invalid codes and offline recovery
Security	Authentication, API access, file permissions and audit integrity
Deployment	Migration failures, service restarts, backup restoration and release rollback

Use Django's testing tools and API tests for the backend, Flutter tests for the mobile application, and end-to-end tests for critical user journeys. Add realistic synthetic data for development and testing rather than copying identifiable production patient data into local environments.

19. Decisions to finalise before coding

Some details in the original specification are not yet defined sufficiently for safe implementation.

Architecture decisions

1. Initial platform scope

- LIS first, then RIS
- LIS and RIS together, in phases

2. UHID generation

- Stable generated UHID, with phone number used for duplicate matching
- UHID derived from the patient's phone number

The stable generated identifier is recommended because phone numbers can change or be shared.

3. Analyser gateway

- Python Windows service with local Bootstrap configuration UI
- Standalone desktop application

4. Radiology/PACS starting point

- Integrate with existing Orthanc/PACS infrastructure

- Plan a new PACS deployment alongside RIS

5. Pilot centre

- Enugu HQ
- NOWAS
- Independence

6. Workflow configuration

- Database-backed drag-and-drop workflow builder
- Predefined workflows first, visual builder later

Final recommendation

Build HFDN's platform as a secure, multi-centre healthcare system with shared patient identity, transactional billing, configurable clinical workflows and separate local integration infrastructure for laboratory and radiology equipment.

The most effective starting point is to establish the Django/PostgreSQL foundation, centre-aware permissions, patient registration, test catalogue and billing before moving into the complete LIS workflow. Keep the code in GitHub, deploy incrementally to Railway, and introduce the analyser gateway only after the central order, sample and result APIs have been tested.

One final point: this is an enterprise-scale clinical software project. A functioning application is not automatically production-ready. Production readiness requires clinical validation, security testing, instrument-specific verification, financial reconciliation, backup recovery and formal operational sign-off before the system is relied on for patient care.